

Computer Networks LAB (CS3P004)
Spring Semester 2024-25
Lab Sheet 5
Socket Programming in C

1) Recall the previous lab on TCP sockets:

a) Server program:

https://drive.google.com/file/d/1ppx6mQDSq_WOjC18qo752c2bqTMFMio9/view?usp=sharing

b) Client Program:

<https://drive.google.com/file/d/16goEugVs9e3kokbj6s8tyKgtWoUwPdZP/view?usp=sharing>

Web Resources: Beej's Guide to Network Programming:
<https://beej.us/guide/bgnet/html/>

2) In this assignment we design the client-server chat programs as follows:

i) Develop a Chat Server program using threads which can:

- (1) Handle multiple clients at the same time.
- (2) A client can join/disconnect from the chat.
- (3) Two clients can chat via server.
- (4) A client can choose to broadcast the message to all clients alive.
- (5) Add more functionalities, like timeout, etc., it is up to your creativity.

ii) Design the Chat Server program (in place of threads) use select() system call to connect multiple clients.

iii) Design the Client program for the chat-server.

Group Details:

S-no	Name	Roll no
1	Borru Vijay Sai	22CS01076
2	G.S.V.Pramod	22CS01033
3	Nitin Sharma	22CS01027
4	Sanjay Kumar Pandit	22CS02012
5	Aayush J Gote	22CS01059

Report:

Readme File:

Have Implemented a client-server chat program using threads which can:

- (1) Handle multiple clients at the same time.
- (2) A client can join/disconnect from the chat.
- (3) Two clients can chat via server.
- (4) A client can choose to broadcast the message to all clients alive.
- (5) Add more functionalities, like timeout(server will disconnect the client who are idle for more than 5 minutes), get the active clients list, response of the msg is it sent or not

2)

i) The server.c code implements a multi-client chat server that allows clients to send messages to specific recipients or to a group. It utilizes threads to handle multiple clients simultaneously, and it ensures thread safety using mutexes. Here's a breakdown of its key components:

1. Client Information:

- The server stores client information (socket, name, IP, and port) in an array of Client structures.
- The clients_mutex mutex is used to manage access to this array, preventing race conditions when accessing or modifying client details.

2. Functions:

- **send_message**: Sends messages to either a specific client or to all active clients (if the recipient is "group").
- **send_active_clients**: Sends the list of active clients to a requesting client.
- **handle_client**: A thread function that handles communication with individual clients:
 - It first receives the client's name and stores it in the client list.
 - The client can send messages using the format Recipient:Message, which is processed and forwarded.

- If the client sends "GET_ACTIVE_USERS", the server sends a list of active users.
- Inactivity or disconnection results in the client being removed from the client list.

3. Main Server Logic:

- The server listens on a specified port and accepts incoming connections.
- For each client connection, it creates a new thread using `pthread_create` to handle the client.
- The `pthread_detach` function is used to automatically clean up threads after they finish executing.

4. Timeout:

- A timeout of 5 minutes (300 seconds) is set for each client's socket, disconnecting clients who have been inactive for that period.

This implementation ensures concurrent communication with multiple clients, supports direct and group messaging, and maintains thread safety and resource management.

2)

ii) The `server_using_select.c` code implements a multi-client chat server that allows clients to send messages to a specific recipient or a group. Unlike the thread-based model, this server uses `select()` to handle multiple clients concurrently, allowing it to monitor multiple file descriptors for input/output without blocking. Below is a breakdown of the key components and functionality:

Key Components:

● Client Structure:

- Stores client details such as socket, name, IP address, and port number in an array of Client structures.

● Active FDs:

- `fd_set active_fds` and `fd_set read_fds` are used to manage the sockets and client connections using `select()` for multiplexing

input/output. This allows the server to handle multiple clients at once without using threads.

Functions:

1. **send_acknowledgment(int sockfd, const char *message):**
 - Sends an acknowledgment message to the client to confirm successful message delivery.
2. **send_message(char *recipient, char *message, char *sender, int sender_socket):**
 - Sends a message to a specific client or broadcasts it to the entire group (if the recipient is "Group").
3. **send_active_clients(int sockfd):**
 - Sends the list of active users to the requesting client.
4. **handle_client_message(int sockfd):**
 - Processes incoming messages from clients. It interprets the message format (Recipient:Message) and forwards it accordingly.
 - It also handles the GET_ACTIVE_USERS command by sending the list of active clients.

Main Server Logic:

- **Socket Initialization:**
 - Creates a socket and binds it to a specified port for listening to incoming connections.
- **Multiplexing with select():**
 - Uses select() to monitor multiple file descriptors, allowing the server to handle multiple clients concurrently without needing multiple threads. This approach provides efficient handling of I/O for all connected clients.
- **Client Connection Handling:**
 - When a new client connects, the server accepts the connection and receives the client's name.
 - The server stores client details (socket, name, IP, port) and adds the client socket to the active_fds set for future communication.
- **Message Handling:**
 - The server receives messages in the format Recipient:Message from clients, forwards the message to the correct recipient, and sends acknowledgments as appropriate.

- If the client sends GET_ACTIVE_USERS, the server responds with a list of all active clients.

- **Client Disconnection:**

- When a client disconnects, the server removes the client from the list and closes the socket, ensuring resources are cleaned up and the client is removed from the active_fds set.

Conclusion:

The server_using_select.c is a multi-client chat server that uses select() for managing multiple client connections without threads. It provides messaging capabilities to send messages to specific users or groups and allows clients to view active users. The server efficiently handles I/O operations using select() to monitor multiple clients simultaneously while maintaining simplicity by not relying on threads.

2)

ii) This client.c program implements a basic multi-client chat system using a client-server model where the client communicates with the server via a socket. It includes multi-threading and uses a message buffer to temporarily store messages while the user is typing. Below is a breakdown of the key components and functionality:

Key Components:

1. Global Variables:

- sockfd: The socket file descriptor used for communication with the server.
- lock: A mutex used to synchronize access to shared resources, ensuring thread safety when multiple threads are involved.
- is_typing: A flag to indicate whether the user is typing a message. It prevents displaying incoming messages while the user is typing.
- skip_prompt_once: A flag to skip the prompt once the acknowledgment (ACK) message is received.

- unread_count: Keeps track of the number of unread messages.
- message_buffer: A circular buffer to store messages while the user is typing or if the message is an acknowledgment.

2. Functions:

- **error()**: Prints an error message and terminates the program.
- **get_time()**: Retrieves the current time in HH:MM:SS format and stores it in the provided buffer.
- **receive_messages()**: A thread function that listens for incoming messages from the server. It:
 - Receives messages and handles different types (error, acknowledgment, active users).
 - Timestamps the messages and stores or displays them.
 - Prompts the user to send a message after displaying incoming messages.
- **main()**: The main client logic that initializes the socket connection and spawns the thread to receive messages. It also handles user input and sends messages to the server.

3. Message Flow:

- **Client Initialization**: When the client connects to the server, it sends the username and awaits further communication.
- **Sending Messages**: The user can send messages to a specific recipient or request a list of active users. If the message is a special command (e.g., "active"), it sends a request to get active users.
- **Message Buffering**: If the user is typing, incoming messages are temporarily buffered and displayed once the user finishes typing.
- **Unread Messages**: If there are unread messages while the user is typing, the program stores them and displays them once the typing session ends.

Flow of Execution:

1. Initialization:

- The program creates a socket and connects to the server using the provided IP address and port.
- It sends the client's username to the server.

2. Receive Messages:

- A thread (recv_thread) continuously listens for messages from the server. It handles error messages, acknowledgment messages, and incoming messages with timestamps. If the user is typing, messages are buffered for later display.

3. User Input:

- The main loop prompts the user to send a message or request the active users list.
- If the user enters a message or command, it is sent to the server. Messages are sent in the Recipient:Message format.
- When the user finishes typing, unread messages are displayed, and acknowledgment messages are processed first.

4. Acknowledgments:

- Once a message is sent, the server may respond with an acknowledgment ("ACK:"). The client will display these messages before any unread messages.

5. Message Storage:

- The program stores up to MAX_BUFFERED_MSGS messages while the user is typing and displays them after the typing session ends.

Conclusion:

The program is a basic, interactive chat client that uses threads for receiving messages and handles multiple clients using socket programming. It also ensures thread safety using a mutex and manages incoming messages while the user is typing. Additionally, it provides a simple mechanism for displaying unread messages and acknowledgment responses.

How it works:

First we will start the server by certain port number and then will connect the clients using the same network to establish a connection between the clients and a common main server. As shown below If any client is been added to the network we will take the ip address of the client and the port number and the name and then the client list is being formed and we can get their ip's by that client list.

```
borru-vijay-sai@borru-vijay-sai-Victus-by-HP-Gaming-Laptop-15-fa0xxx:~/Desktop/C
N/group_lab$ gcc final_server.c -o server
borru-vijay-sai@borru-vijay-sai-Victus-by-HP-Gaming-Laptop-15-fa0xxx:~/Desktop/C
N/group_lab$ ./server 5000
Main Server listening on port 5000...
Vijay connected from IP: 10.10.92.129
Nitin connected from IP: 10.42.0.19
sanjay connected from IP: 10.42.0.10
█
```

```
borru-vijay-sai@borru-vijay-sai-Victus-by-HP-Gaming-Laptop-15-fa0xxx:~/Desktop/C
N/group_lab$ gcc final_client.c -o client
borru-vijay-sai@borru-vijay-sai-Victus-by-HP-Gaming-Laptop-15-fa0xxx:~/Desktop/C
N/group_lab$ ./client 10.10.92.129 5000 Vijay
Connected as Vijay. Use format 'Recipient:Message' to chat. Type 'active' to get
the list of online users.
Do you want to send a message? (y): █
```

The client will work as it asks to give input 'y' if needs to send a message to his friends or broadcast in group. We need to follow a format of Recipients Name: The msg to be sent. After successfully sending we will get the Acknowledgment after sending as successful or failed with time stamps.

```
Connected as Vijay. Use format 'Recipient:Message' to chat. Type 'active' to get
the list of online users.
Do you want to send a message? (y): y
You: Nitin:Hello I'm Vijay

[23:10:59] Message delivered successfully
Do you want to send a message? (y): y
You: sanjay:Hello I'm Vijay

[23:11:29] Message delivered successfully
Do you want to send a message? (y): y
You: Group:Hello All is my message reachable to all of u

[23:11:55] Message delivered to group.
Do you want to send a message? (y): █
```

```

N/group_lab$ ./server 5000
Main Server listening on port 5000...
Vijay connected from IP: 10.10.92.129
Nitin connected from IP: 10.42.0.19
sanjay connected from IP: 10.42.0.10
Message from Vijay to Nitin: Hello I'm Vijay
Message from Vijay to sanjay: Hello I'm Vijay
Message from Vijay to Group: Hello All is my message reachable to all of u
Message from Nitin to Vijay: Hello man, how are u?
Message from Vijay to Nitin: I'm Fine. How about you?
Message from Nitin to sanjay: What are u doing?
sanjay disconnected
sanjay connected from IP: 10.42.0.10
Message from sanjay to Vijay: how do you do?
Message from Vijay to Nitin: Not read the message??

Nitin disconnected due to inactivity

sanjay disconnected due to inactivity

Vijay disconnected due to inactivity
□

```

```

You: sanjay:Hello I'm Vijay

[23:11:29] Message delivered successfully
Do you want to send a message? (y): y
You: Group:Hello All is my message reachable to all of u

[23:11:55] Message delivered to group.
Do you want to send a message? (y):
[23:12:48] Nitin: Hello man, how are u?
Do you want to send a message? (y): y
You: Nitin:I'm Fine. How about you?

[23:13:29] Message delivered successfully
Do you want to send a message? (y): y
You: Nitin:Not read the message??

==== NEW UNREAD MESSAGES ====
[23:16:43] sanjay: how do you do?
==== END OF UNREAD MESSAGES ====

[23:17:08] Message delivered successfully
Do you want to send a message? (y):
Disconnected from server.
█

```

If two persons are typing then to not disturb the flow of message we are buffering the messages separately until the typed msg is been sent then we

will display them as unread msgs as shown above.

From other client devices:

```
o nitin@Ubuntu:~/Desktop/cn_assign$ ./client 10.10.92.129 5000 Nitin
Connected as Nitin. Use format 'Recipient:Message' to chat. Type 'active' to get the list of online users.
Do you want to send a message? (y):
[23:10:58] Vijay: Hello I'm Vijay
Do you want to send a message? (y):
[23:11:54] Vijay: Hello All is my message reachable to all of u
Do you want to send a message? (y): y
You: Vijay: Hello man, how are u?

[23:12:47] Message delivered successfully
Do you want to send a message? (y): y
You: sanjay: What are u doing?

==== NEW UNREAD MESSAGES ====
[23:13:28] Vijay: I'm Fine. How about you?
==== END OF UNREAD MESSAGES ====

[23:14:13] Message delivered successfully
Do you want to send a message? (y):
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
sanjay-pandit@SANJAY-GRAVITON:~/Documents/22cs02012_a5$ gcc client.c -o client
sanjay-pandit@SANJAY-GRAVITON:~/Documents/22cs02012_a5$ ./client 10.10.92.129 5000 sanjay
Connected as sanjay. Use format 'Recipient:Message' to chat. Type 'active' to get the list of online users.
Do you want to send a message? (y):
[23:11:29] Vijay: Hello I'm Vijay
Do you want to send a message? (y):
[23:11:55] Vijay: Hello All is my message reachable to all of u
Do you want to send a message? (y):
[23:14:14] Nitin: What are u doing?
Do you want to send a message? (y):
```

Additional Functionalities:

Implemented a timeout function as if there is no messages and no responses sent to a particular client then the server will automatically remove the clients connection from the network.

```
Nitin disconnected due to inactivity
sanjay disconnected due to inactivity
^C
```

Implemented a new function as to get the list of active users at that time :

```
Do you want to send a mes
You: active

[00:02:25] Active Users:
Vijay
Sai
```